

PCM Skills 之外，适合 LabGuardian 的前沿技术补强清单

你们当前 PCM Skills 的定位与“同族技术”的判据

你们在初选方案里已经把 PCM (Push-Based Context Management) 技能挂载定义为“CPU 侧意图控制面 + Skills 动态路由 + Context Compiler 上下文实时编译”，核心目标是：在边缘端上下文预算极小、算力异构 (CPU/iGPU/NPU) 且安全关键 (电气诊断) 场景下，让系统具备“无限技能库但单次推理低 Token”的可扩展性与可解释性。¹

因此，“类似 PCM 的前沿技术”我建议用同一套判据筛选：

能否同时提升 (1) **可控性** (可解释、可审计、可回滚)、(2) **可靠性** (少幻觉、可验证)、(3) **低成本** (少 Token、少外部依赖)、(4) **工程易落地** (能在 FastAPI/Celery/WS 架构里快速上线) —— 这四点越占优势，越适合竞赛场景。

下文给出我认为与你们 PCM 互补且“评审可讲清楚、可量化展示”的 6 类前沿技术方向，并附上你们可以在 PPT 中讲的“价值句式”。

面向 agent 的“可控行动式推理”技术

ReAct：把“推理痕迹”与“工具行动”交织生成

ReAct 的关键贡献是：让模型在生成过程中显式产生“思考/推理痕迹 (Reasoning traces)”与“行动 (Actions)”，并通过行动调用外部知识接口来减少幻觉与误差传播，同时提升可解释性与可追溯性。¹

为什么适合你们：

你们很多“诊断”本来就分阶段 (拓扑发现→检索 datasheet→下发排故 skill→生成结构化报告)。PCM 已经解决“技能注入”与“上下文编译”，ReAct 补上的是：把每次 agent 的决策过程标准化成**可展示的可解释轨迹** (评委很喜欢)。¹

PPT 价值句式：

“我们不是让大模型直接给结论，而是让它**按 ReAct 路线：先基于拓扑证据做推理→再触发工具 (RAG/规则/热力图)→再生成结论**，每一步可审计可复现。”

Toolformer 思路：让“何时调用什么工具”从规则走向可学习

Toolformer 提出了一种自监督方式，让语言模型学会：何时调用工具、调用哪个工具、传什么参数、如何利用返回结果，从而在多任务上增强能力且不牺牲语言建模能力。²

为什么适合你们：

你们现在的 Skill Router 更偏规则 (意图→技能子集)。Toolformer 可以作为“**学习型 Router**”的理论依据：用你们真实课堂日志 (错误类型、学生反馈、最终修复结果) 构造训练样本，让一个小模型/轻量策略网络学会更优的技能组合选择 (例如在某些症状下优先推送“引脚排故”而不是“热力图”)。这能显著提升“指导有效性”指标，同时又不会引入巨大推理成本。³

PPT 价值句式：

“我们将技能调度从纯规则升级为 **可学习的技能路由器**：通过历史纠错数据自动学习‘最少的工具调用达到最高的纠错成功率’。”

面向 RAG 的“可靠性增强”技术

Self-RAG：检索按需触发 + 自反思，降低“无必要检索/误用检索”带来的劣化

Self-RAG指出传统 RAG 的问题：不分场景固定 top-k 检索，可能引入无关证据、降低模型多样性，甚至带来错误。Self-RAG 用“按需检索 + 自反思（reflection tokens）”提升事实性与回答质量。 ⁴

为什么适合你们：

你们做的是“安全关键诊断”（电气规则、绝对最大额定值、引脚定义）。Self-RAG 的价值在于：当拓扑证据已经足够（例如明确短路/极性反接），**不必检索**，直接走规则/拓扑→输出；只有在“拓扑正确但异常”或“型号未知/参数缺失”时再检索 datasheet/经验案例，从而减少 Token、降低检索噪声、提升稳定性。 ⁵

PPT 价值句式：

“我们不是每次都检索，而是 **Self-RAG 按需检索**：把检索当作工具，只有在证据不足时触发，从而成本更低、回答更稳。”

HyDE：没有标注也能更准的零样本检索（尤其适合你们 datasheet/手册语料）

HyDE (Hypothetical Document Embeddings) 提出：先让模型生成“假想相关文档”，再用该文档的 embedding 去找真实语料邻域，从而在无相关性标注的情况下提升零样本 dense retrieval。 ⁶

为什么适合你们：

芯片手册/实验指导书这类语料往往术语密集、表达方式与学生自然语言不一致。HyDE 可以作为“查询改写/语义桥接”的低成本方案：把学生语言（“我怀疑短路”“运放不输出”）转成更像 datasheet 语域的“假想段落”，再做向量检索，召回更准。 ⁶

PPT 价值句式：

“我们用 HyDE 做‘查询语域转换’，把学生自然语言变成 datasheet 风格语义，再检索，显著提高召回率。”

“图结构增强检索”方向：天然契合你们的电路拓扑与网表

GraphRAG / GRAG：用知识图谱/文本图进行多跳检索与整体性推理

你们系统的核心资产之一是“从图像→拓扑→SPICE 网表→规则校验”，天然就是**图结构**。GraphRAG 方向的关键洞见是：仅靠文档 chunk 的近邻检索（naive RAG）很难捕捉跨文档、多跳关系；引入图结构（实体-关系-社区）可提升对抽象概念与多跳问题的覆盖与综合能力。 ⁷

更具体地，GRAG (Graph Retrieval-Augmented Generation) 提出针对“网络化文档”的子图检索与图上下文增强生成框架，在需要多跳推理的任务上显著优于传统 RAG。 ⁸

为什么适合你们（非常关键）：

- 你们的“对象”不仅是文本 chunk，而是 **（芯片型号）—（引脚）—（典型电路）—（故障模式）—（排查步骤）**的关系网络；

- 你们还有一张实时生成的“电路连接图/网表图”。

把这两类图连起来，就能做出“评审一看就懂”的高级能力：

从当前网表的异常边/节点出发 → 在知识图谱中做局部子图检索 → 返回最相关的引脚规则/典型错误/修复步骤。 9

PPT 价值句式：

“我们把 RAG 升级为 GraphRAG：用‘电路网表图 + 芯片知识图’做子图级检索，实现多步推理与更强覆盖，同时降低幻觉。”

工程落地层面的“前沿但可迅速见效”的补强

有状态的 agent 编排与人类在环：LangGraph 类“可视化状态机”思路

LangGraph 强调为长任务、可中断、可恢复、带 streaming 与 human-in-the-loop 的 agent 提供底层编排能力（更像“有状态图执行引擎”而不是一次性 prompt）。 10

为什么适合你们：

你们的诊断链路本质是多步、可回放的：感知→重构→规则→检索→技能→报告，而且要面向课堂现场的稳定演示。把 agent 逻辑显式建模为状态机/有向图（甚至直接用 mermaid 用于展示），会显著提升“工程成熟度观感”：可暂停、可重试、可回滚、可插入教师确认（人类在环）。 11

PPT 价值句式：

“我们的 agent 不靠黑盒循环，而是用‘可恢复的状态机编排’，现场更稳定、更可控。”

安全与可信：把“幻觉风险”变成可验证机制（对评审非常加分）

你们属于安全关键场景（电气参数、额定值、错误建议可能导致硬件损坏）。建议把“可信”设计成机制而不是口号：

- **必须引用证据**：RAG 结果必须携带出处（datasheet 页码/段落 ID、知识库文档 ID）；
- **关键字段结构化输出 + 校验**：把“引脚号/额定值/电压范围/风险等级”作为结构化字段，先由规则引擎校验，再生成自然语言解释；
- **对外部行动做审计**：每次技能调用、检索、生成都写审计日志，支持复盘与归责。

这些做法与“GraphRAG + ReAct + PCM”组合后，会在评审中形成一个非常强的叙事：**我们做的是可审计的工程系统，不是聊天机器人。** 12

建议你们优先加入的“三个前沿点”与集成顺序

结合你们已经规划的 PCM+RAG+异构推理路线，我建议按“投入产出比”排序：

1) ReAct 风格的可解释行动轨迹（高性价比、评审观感强）：

先把现有 pipeline/skills/RAG 都包装成 actions，输出“步骤-证据-结论”链路。 1

2) GraphRAG（把网表图与知识图合并，形成你们的独特壁垒）：

这是最贴合你们电路任务的差异化点，且很容易做成 demo：同一错误，naive RAG 给片段；GraphRAG 给“局部子图 + 证据链 + 建议”。 13

3) Self-RAG/HyDE（把成本、稳定性做成可量化指标）：

以“平均检索次数/平均 Token/平均时延/引用正确率”为指标在 PPT 里对比，能直接转化为评审打分的技术优势。 14

如果你愿意，我可以按你们 DK-2500 的离线约束，把这三项具体落到你们现有 FastAPI/Celery/WebSocket 的接口与模块分层（哪一层做图谱构建、哪一层做检索、哪一层做技能路由、哪些结果作为可视化证据链输出）。

1 12 ReAct: Synergizing Reasoning and Acting in Language Models - arXiv.gg

https://arxiv.gg/abs/2210.03629?utm_source=chatgpt.com

2 3 Toolformer: Language Models Can Teach Themselves to Use Tools - arXiv.gg

https://arxiv.gg/abs/2302.04761?utm_source=chatgpt.com

4 Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection - arXiv.gg

https://arxiv.gg/abs/2310.11511?utm_source=chatgpt.com

5 14 Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection for ICLR 2024 - IBM Research

https://research.ibm.com/publications/self-rag-learning-to-retrieve-generate-and-critique-through-self-reflection?utm_source=chatgpt.com

6 Precise Zero-Shot Dense Retrieval without Relevance Labels - arXiv.gg

https://arxiv.gg/abs/2212.10496?utm_source=chatgpt.com

7 13 Project GraphRAG - Microsoft Research

https://www.microsoft.com/en-us/research/project/graphrag/%3Flang%3Dzh-cn?utm_source=chatgpt.com

8 GRAG: Graph Retrieval-Augmented Generation | GraphRAG

https://graphrag.com/appendices/research/2405.16506/?utm_source=chatgpt.com

9 Retrieval-Augmented Generation with Graphs (GraphRAG) | GraphRAG

https://graphrag.com/appendices/research/2501.00309/?utm_source=chatgpt.com

10 LangGraph overview - Docs by LangChain

https://docs.langchain.com/oss/python/langgraph/overview?utm_source=chatgpt.com

11 LangGraph overview - Docs by LangChain

https://docs.langchain.com/oss/javascript/langgraph/overview?utm_source=chatgpt.com